Section 18 : Exception Handling

Information

- Title: `Exception Handling`
- Authors: `Etienne KOA`

Aperçu de la section

- Qu'est-ce qu'une exception ?
- Qu'est-ce qu'une gestion des exceptions ?
- Que lance-t-on et intercepte-t-on les exceptions ?
- Comment cela affecte-t-il le flux de contrôle ?
- Définir nos propres classes d'exceptions
- La bibliothèque d'exceptions de la bibliothèque standard
  - `std::exception` et `what()`

Concepts de base et un exemple simple : Division par zéro

- Gestion des exceptions
  - faire face à des situations extraordinaires
  - indique qu'une situation extraordinaire a été détectée ou s'est produite
  - le programme peut gérer les situations extraordinaires de manière appropriée
- Qu'est-ce qui cause les exceptions ?
  - ressources insuffisantes
  - ressources manquantes
  - opérations invalides
  - violations de portée
  - débordements et débordements
  - Données illégales et autres
- Coffre-fort d'exception
  - Quand votre code gère les exceptions

Syntaxe `C++`

- **`lancer`**
  - lève une exception
  - suivi d'un argument
- **`try {code pouvant lever une exception}`**
  - vous placez du code qui peut lever une exception dans un bloc try
  - si le code lève une exception, le bloc try est quitté
  - l'exception levée est gérée par un gestionnaire catch
  - si aucun gestionnaire catch n'existe, le programme se termine
- **`catch {Exception ex} {code pour gérer l'exception}`**
  - code qui gère l'exception
  - peut avoir plusieurs gestionnaires de capture

  - peut ou non entraîner la fin du programme

Diviser par zéro exemple

- Que se passe-t-il si le « total » est nul
  - crash, débordement ?
  - ça dépend

```
double average {};
average = sum / total;
```

```
double average {};
if (total == 0)
   // what to do?
else
   average = sum / total;
```

```
double average {};
try {                       // try block
   if (total == 0)
      throw 0;              // throw the exception
   average = sum / total;   // won't execute if total == 0
   // use average here
}
catch (int &ex) {           // exception handler
   std::cerr << "can't divide by zero" << std::endl;
}
std::cout << "program continues" << std::endl;
```

Lancer une exception à partir d'une fonction

- Lancer une exception à partir d'une fonction
- Que renvoie-t-on si le total est nul ?

```
double calculate_avg(int sum, int total) {
   return static_cast<double>(sum) / total;
}
```

Figure 1: Exception

- Lancer une exception à partir d'une fonction

- Lancez une exception si nous ne pouvons pas terminer avec succès

```
double calculate_avg(int sum, int total) {
   if (total == 0)
      throw 0;
   return static_cast<double>(sum) / total;
}
```

Figure 2: Exception

La capture d'une exception a lancé une fonction

```
double average {};

try {
   average = calculate_avg(sum, total);
   std::cout << average << std::endl;
}
catch (int &ex) {
   std::cerr << "You can't divide by zero" << std::endl;
}

std::cout << "Bye" << std::endl;
```

Figure 3: Exception

Gestion des exceptions

- Lancer une exception à partir d'une fonction
- Que se passe-t-il si une fonction peut échouer de plusieurs manières
  - `gallons` vaut zéro
  - `miles` ou `gallons` est négatif

```
double calculate_mpg(int miles, int gallons) {
   return static_cast<double>(miles) / gallons;
}
```

Figure 4: Exception

- Lancer une exception à partir d'une fonction
- Lancez des exceptions de type différent pour chaque condition

```
double calculate_mpg(int miles, int gallons) {
   if (gallons == 0)
      throw 0;
   if (miles < 0 || gallons < 0)
      throw std::string{"Negative value error"};

   return static_cast<double>(miles) / gallons;
}
```

Figure 5: Exception

```
double miles_per_gallon {};
try {
   miles_per_gallon = calculate_mpg(miles, gallons);
   std::cout << miles_per_gallon << std::endl;
}
catch (int &ex) {
   std::cerr << "You can't divide by zero" << std::endl;
}
catch (std::string &ex) {
   std::cerr << ex << std::endl;
}

std::cout << "Bye" << std::endl;
```

Figure 6: Exception

La capture d'une exception a lancé une fonction

Attraper tout type d'exception

```
catch (int &ex) {
}
catch (std::string &ex) {
}
catch (...) {
   std::cerr << "Unknown exception" << std::endl;
}
```

Figure 7: Exception

Stacking Unwiding et comment ça marche

Empilage et déroulement

Si une exception est levée mais n'est pas interceptée dans la portée actuelle, « C++ » essaie de trouver un gestionnaire pour en déroulant la pile.

- La fonction dans laquelle l'exception n'a pas été interceptée se termine et est supprimée de la pile d'appels.
- Si un bloc try a été utilisé, les blocs catch sont vérifiés pour une correspondance.
- Si aucun bloc try n'a été utilisé ou si le gestionnaire catch ne correspond plus au déroulement du processus.
- Si la pile est déroulée vers main et qu'il n'y a pas de gestionnaire catch, le programme se termine.

Création de classes d'exception définies par l'utilisateur

Exceptions définies par l'utilisateur

- Nous pouvons créer des classes d'exception et lancer des instances de ces classes
- Bonne pratique :
  - lancer un objet pas un type primitif
  - lancer un objet par valeur
  - attraper un objet par référence (ou référence `const`)

Création de classes d'exception

```
class DivideByZeroException {

} ;

class NegativeValueException {

} ;
```

Lancement de classes d'exception définies par l'utilisateur

```
double calculate_mpg(int miles, int gallons) {

   if (gallons == 0)
      throw DivideByZeroException();
   if (miles < 0 || gallons < 0)
      throw NegativeValueException();

   return static_cast<double>(miles) / gallons;
}
```

Figure 8: Exception

Capture des exceptions définies par l'utilisateur

```
try {
   miles_per_gallon = calculate_mpg(miles, gallons);
   std::cout << miles_per_gallon << std::endl;
}
catch (const DivideByZeroException &ex) {
   std::cerr << "You can't divide by zero" << std::endl;
}
catch (const NegativeValueException &ex) {
   std::cerr << "Negative values aren't allowed" << std::endl;
}

std::cout << "Bye" << std::endl;
```

Figure 9: Exception

Exceptions au niveau de la classe

Des exceptions peuvent également être levées depuis l'intérieur d'une classe :

- Méthode
  - Ceux-ci fonctionnent de la même manière que pour les fonctions comme nous l'avons vu

- Constructeur
  - Les constructeurs peuvent échouer
  - Les constructeurs ne renvoient aucune valeur
  - Lancez une exception dans le constructeur si vous ne pouvez pas initialiser un objet
- Destructeur
  - Ne lancez PAS d'exceptions de votre destructeur

Exceptions au niveau de la classe

```
Account::Account(std::string name, double balance)
    : name{name}, balance{balance} {

   if (balance < 0.0)
      throw IllegalBalanceException{};

}

try {
   std::unique_ptr<Account> moes_account =
        std::make_unique<Checking_Account>("Moe",-10.0);
   // use moes_account
}
catch (const IllegalBalanceException &ex) {
   std::cerr << "Couldn't create account" << std::endl;
}
```

La hiérarchie des classes `C++` `std::exception`

- La hiérarchie des classes d'exceptions de la bibliothèque standard `C++`
- `C++` fournit une hiérarchie de classes de classes d'exceptions
  - `std::exceptions` est la classe de base
  - toutes les sous-classes implémentent la fonction virtuelle `what()`
  - nous pouvons créer nos propres sous-classes d'exception définies par l'utilisateur

```
virtual const char * what() const noexecpt;
```

La hiérarchie des classes d'exceptions de la bibliothèque standard `C++`

Dériver notre classe de `std::exception`

Notre constructeur de classe de compte modifié

Création d'un objet Compte

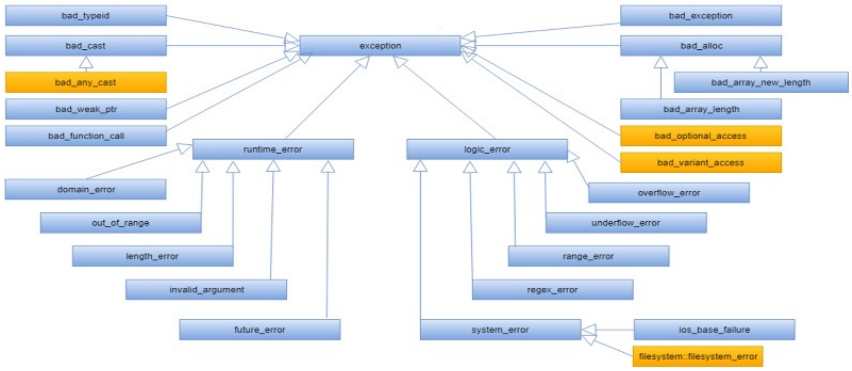

Figure 10: Exception

```
class IllegalBalanceException: public std::exception
{
public:
   IllegalBalanceException() noexcept = default;
   ~IllegalBalanceException() = default;
   virtual const char* what() const noexcept {
      return "Illegal balance exception";
   }
};
```

Figure 11: Exception

```
Account::Account(std::string name, double balance)
    : name{name}, balance{balance} {
    if (balance < 0.0)
       throw IllegalBalanceException{};
}
```

Figure 12: Exception

```
try {
   std::unique_ptr<Account> moes_account =
      std::make_unique<Checking_Account>("Moe", -100.0);

   std::cout << "Use moes_account" << std::endl;
}
catch (const IllegalBalanceException &ex)
{
   std::cerr << ex.what() << std::endl;
         // displays "Illegal balance exception"
}
```

Figure 13: Exception